MODULE 02

Functional Programming in Python

Pure functions, higher-order functions, iterators, generators and decorators

```
>>> sum(n * n for n in range(10) if n % 2)
```

AIN3005 · Advanced Python Programming

Binnur Kurt, PhD · Bahçeşehir University · AI Engineering



MODULE 02 · FUNCTIONAL PROGRAMMING

Learning Objectives

By the end of this module, you will be able to:

1. Trace function calls on the call stack and explain what a stack frame stores
2. Distinguish pure from impure functions and avoid hidden side effects
3. Pass, return and store functions, and sort data with `key` functions
4. Transform data with `map`, `filter`, `reduce` and comprehensions
5. Write iterators and generators and build lazy pipelines with `itertools`
6. Implement closures and decorators, including decorators with arguments
7. Apply `functools.partial`, `cache`/`lru_cache` and `cached_property`
8. Refactor an imperative data-processing script into a functional pipeline

MODULE 02 · FUNCTIONAL PROGRAMMING

Module Roadmap

01 **Functions and the Call Stack**
What happens in memory when a function is called

02 **Functional-Style Programming**
Say what you want, not how to compute it

03 **First-Class and Higher-Order Functions**
Functions are values you can pass around

04 **Map, Filter, Reduce and Comprehensions**
Transforming collections declaratively

05 **Iterators and Generators**
Producing values lazily, one at a time

06 **Closures and Decorators**
Functions that remember and functions that wrap

07 **The functools Toolbox**
Ready-made higher-order functions and decorators

08 **Case Study: A Functional Data Pipeline**
Refactoring the employee report

01

Functions and the Call Stack

What happens in memory when a function is called

- The function-call stack and stack frames
- Local variables and their lifetime
- Recursion and the recursion limit

The Function-Call Stack

- A **stack** is last-in, first-out (LIFO)
- Each call **pushes** a **stack frame** (activation record)
  - holds parameters, local variables and the return location
- `return` **pops** the frame; control goes back to the caller

```python
def square(x):
    return x * x

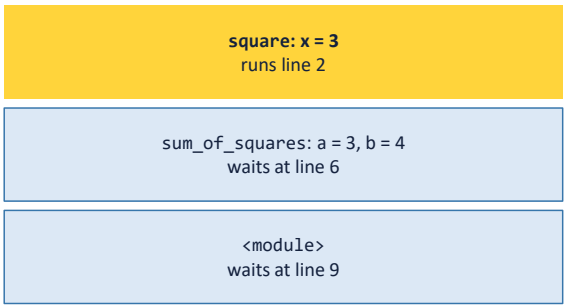

def sum_of_squares(a, b):
    return square(a) + square(b)


print(sum_of_squares(3, 4))
```

OUTPUT

```
25
```

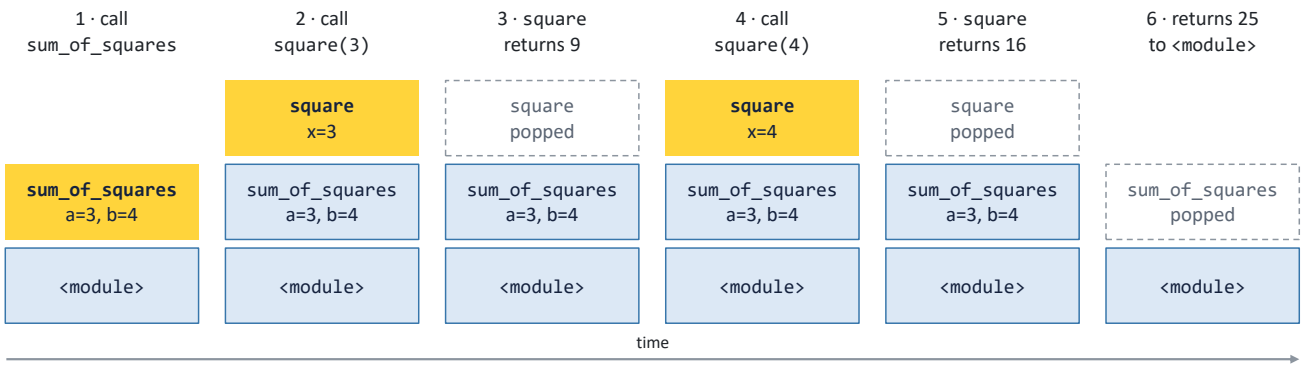
Call stack while line 2 runs



Note

A traceback is a printout of this stack at the moment an exception is raised.

Watching the Stack Grow and Shrink



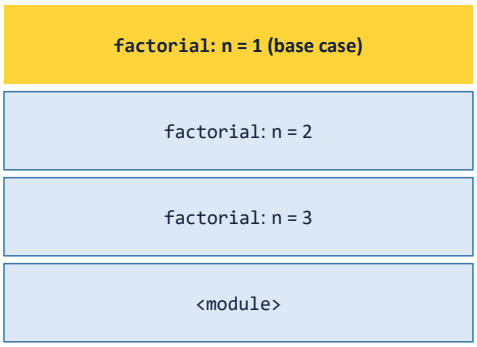
Key idea

Local variables live in their frame: when `square` returns, its `x` is gone. The next call gets a brand-new frame and a new `x`.

Recursion and the Stack Limit

```
>>> def factorial(n):
...     """Return n! recursively."""
...     if n <= 1:
...         return 1
...     return n * factorial(n - 1)
...
>>> factorial(5)
120
>>> import sys
>>> sys.getrecursionlimit()
1000
>>> factorial(5_000)
Traceback (most recent call last):
  ...
RecursionError: maximum recursion depth exceeded
```

Stack at the base case of `factorial(3)`



Watch out

CPython has no tail-call optimisation: deep iteration belongs in a loop.

TRACE THE CODE

Trace the Calls

```
 1  def greet(name):
 2      print(f"enter greet({name})")
 3      message = shout(name)
 4      print(f"leave greet: {message}")
 5      return message
 6
 7
 8  def shout(word):
 9      print(f"enter shout({word})")
10      return word.upper() + "!"
11
12
13  greet("ada")
14  print(message)
```

- Predict every line of output
- How many frames are on the stack at the deepest point?
- What happens on line 14, and why?

ANSWER

Trace the Calls

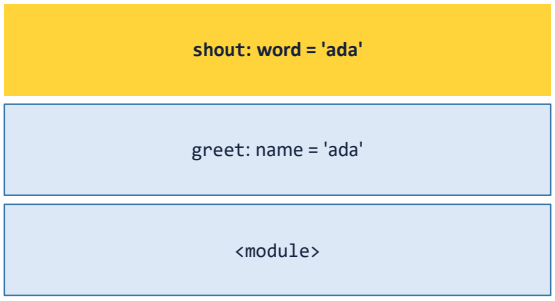
OUTPUT

```
enter greet(ada)
enter shout(ada)
leave greet: ADA!
Traceback (most recent call last):
  File "calls.py", line 14, in <module>
    print(message)
          ^^^^^^^
NameError: name 'message' is not defined
```

Key idea

`message` was a local of `greet`. Its frame was popped at `return`, so the name does not exist at module level.

Deepest point: 3 frames



02

Functional-Style Programming

Say what you want, not how to compute it

- Python is multi-paradigm: functional *style*, not a purely functional language
- Internal vs. external iteration
- Pure functions, side effects and immutability

What vs. How: Internal Iteration

External iteration

you manage the counter

```
total = 0
i = 0
while i < len(prices):
    total += prices[i]
    i += 1
```

Loop over the iterable

Python hides the counter

```
total = 0
for price in prices:
    total += price
```

Internal iteration

say *what* you want

```
total = sum(prices)
```

Key idea

Every variable you mutate is a place for a bug: wrong start value, wrong condition, forgotten increment. Library functions such as `sum`, `min` and `max` own the loop for you.

Pure Functions

- A **pure function** returns a result that depends *only* on its arguments
- Same arguments → same result, every time
- No **side effects**: nothing outside the function changes
- Easy to test, cache, reuse and run in parallel

Key idea

A pure call can be replaced by its result: in this program, `sum(values)` *is* 108.

```
>>> values = [4, 8, 15, 16, 23, 42]
>>> sum(values)
108
>>> sum(values)
108
>>> sum(values)
108
>>> def average(numbers):
...     return sum(numbers) / len(numbers)
...
>>> average(values)
18.0
```

Side Effects and Impure Functions

Impure: hidden input and output

```
tax_rate = 0.18
audit_log = []


def price_with_tax(price):
    audit_log.append(price)  # mutates
    return price * (1 + tax_rate)
```

Pure: everything in, one value out

```
def price_with_tax(price, tax_rate):
    return price * (1 + tax_rate)


print(price_with_tax(100, 0.18))
print(price_with_tax(100, 0.18))
```

OUTPUT

```
118.0
118.0
```

Mutation
global variables, arguments, attributes

Input/output
`print`, files, network, databases

Hidden input
the clock, random numbers, environment

QUIZ

Quick Check: Which Function Is Pure?

Which of these functions is **pure**?

A `def area(r): return math.pi * r ** 2`

B `def log(msg): print(msg)`

C `def stamp(): return datetime.now()`

D
```
def push(item, stack):
stack.append(item)
```

Immutability: Values That Cannot Change

Mutable	Immutable counterpart
`list`	`tuple`
`set`	`frozenset`
`dict`	`types.MappingProxyType` (read-only view)
ordinary class	`@dataclass(frozen=True)`, `NamedTuple`

Key idea

Immutable objects can be shared freely: no function can change them behind your back.

```python
from dataclasses import dataclass, replace


@dataclass(frozen=True)
class Point:
    x: float
    y: float


origin = Point(0, 0)
moved = replace(origin, x=5)  # new object
print(origin, moved)
origin.x = 1
```

OUTPUT

```
Point(x=0, y=0) Point(x=5, y=0)
Traceback (most recent call last):
  ...
dataclasses.FrozenInstanceError: cannot assign to field 'x'
```


FIND THE BUG

Find the Bug: The Cart That Remembers

```python
def add_item(item, cart=[]):
    """Add item to cart and return the cart."""
    cart.append(item)
    return cart


print(add_item("apple"))
print(add_item("pear"))
```

OUTPUT

```
['apple']
['apple', 'pear']
```

- The second call should print `['pear']`
- Why does the function remember `apple`?
- Fix `add_item` without changing how it is called

ANSWER

Find the Bug: The Cart That Remembers

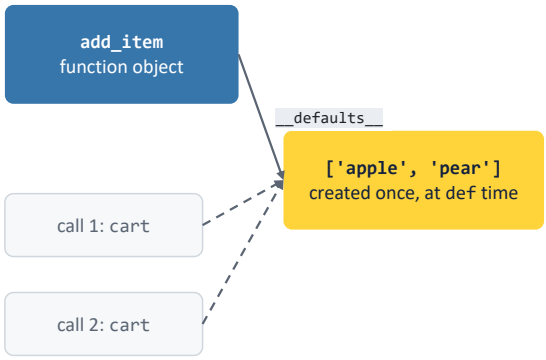
```python
def add_item(item, cart=None):
    """Add item to cart and return the cart."""
    if cart is None:
        cart = []
    cart.append(item)
    return cart


print(add_item("apple"))
print(add_item("pear"))
```

OUTPUT

```
['apple']
['pear']
```

Why the original failed:



Key idea

Default values are evaluated **once**, when `def` runs. Never use a mutable default: use `None` and create the object inside the function.

03

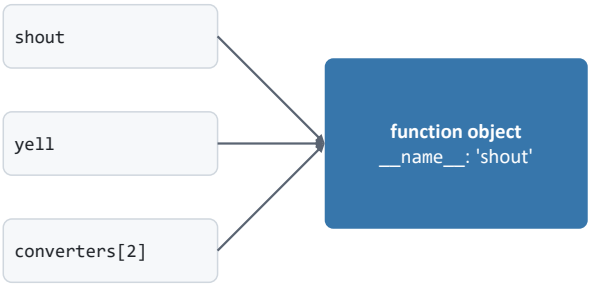
First-Class and Higher-Order Functions

Functions are values you can pass around

- Functions as objects
- Passing and returning functions
- Lambda expressions
- Key functions and the operator module

Functions Are First-Class Objects

- `def` creates a **function object** and binds a name to it
- Functions can be assigned, stored, passed and returned like any value




```
>>> def shout(text):
...     return text.upper() + "!"
...
>>> type(shout)
<class 'function'>
>>> yell = shout
>>> yell("hello")
'HELLO!'
>>> yell.__name__
'shout'
>>> converters = [str.lower, str.title, shout]
>>> [convert("hi you") for convert in converters]
['hi you', 'Hi You', 'HI YOU!']
```


Higher-Order Functions

A **higher-order function** takes a function as an argument or returns a function

Takes a function

```
def for_each(values, action):
    """Call action(value) for each value."""
    for value in values:
        action(value)


numbers = (4, 8, 15, 16, 23, 42)
for_each(numbers, print)
```

OUTPUT

```
4
8
15
16
23
42
```

Returns a function

```
def make_multiplier(factor):
    """Return a multiplier function."""
    def multiply(value):
        return value * factor
    return multiply


double = make_multiplier(2)
print(double(21))
print(make_multiplier(3)(14))
```

OUTPUT

```
42
42
```

Lambda Expressions

`lambda parameters: expression` creates an anonymous function whose body is one expression

	`def`	`lambda`
Name	yes	anonymous (`<lambda>`)
Body	statements	one expression
Docstring, hints	yes	no
Typical use	reusable logic	short inline argument

Watch out

Do not assign a lambda to a name; write a `def` instead (PEP 8).

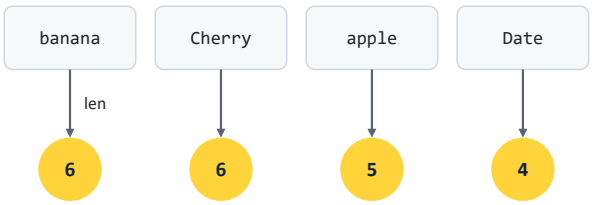
```
>>> numbers = [4, 8, 15, 16, 23, 42]
>>> list(filter(lambda n: n % 2 == 1, numbers))
[15, 23]
>>> (lambda a, b: a * b)(6, 7)
42
>>> pairs = [(3, "three"), (1, "one"), (2, "two")]
>>> sorted(pairs, key=lambda pair: pair[1])
[(1, 'one'), (3, 'three'), (2, 'two')]
```

Tip

`key=lambda s: len(s)` is just `key=len`: pass existing functions directly.

Sorting and Selecting with key Functions

```
>>> words = ["banana", "Cherry", "apple", "Date"]
>>> sorted(words)
['Cherry', 'Date', 'apple', 'banana']
>>> sorted(words, key=str.lower)
['apple', 'banana', 'Cherry', 'Date']
>>> sorted(words, key=len)
['Date', 'apple', 'banana', 'Cherry']
>>> max(words, key=len)
'banana'
>>> scores = {"ada": 91, "alan": 78, "grace": 95}
>>> max(scores, key=scores.get)
'grace'
>>> sorted(scores, key=scores.get, reverse=True)
['grace', 'ada', 'alan']
```



compare the keys, return the words:
Date, apple, banana, Cherry

Key idea

`key` is called once per element; elements are ordered by their keys, but the **elements** are returned.

operator.itemgetter and attrgetter

```python
from dataclasses import dataclass
from operator import attrgetter, itemgetter


@dataclass
class Student:
    name: str
    grade: int


rows = [("grace", 95), ("alan", 78), ("ada", 95)]
print(sorted(rows, key=itemgetter(1)))

students = [Student(name, grade) for name, grade in rows]
ranked = sorted(students, key=attrgetter("grade", "name"))
print([student.name for student in ranked])
```

OUTPUT

```
[('alan', 78), ('grace', 95), ('ada', 95)]
['alan', 'ada', 'grace']
```

Getter	Same as
`itemgetter(1)`	`lambda r: r[1]`
`itemgetter("price")`	`lambda d: d["price"]`
`itemgetter(0, 2)`	`lambda r: (r[0], r[2])`
`attrgetter("grade")`	`lambda s: s.grade`
`methodcaller("lower")`	`lambda s: s.lower()`

TRY IT

Rank the Products

```python
products = [("laptop", 1200), ("mouse", 25), ("monitor", 300)]
```

1. Print the cheapest product using `min`
2. Print the product names from most to least expensive
3. Write `make_discount(rate)` returning a function: `make_discount(0.1)(300)` gives `270.0`

> **Tip**
>
> Use `key=` and `operator.itemgetter`; no explicit `for` loop is needed.

ANSWER

Rank the Products

```python
from operator import itemgetter

products = [("laptop", 1200), ("mouse", 25), ("monitor", 300)]
by_price = itemgetter(1)
print(min(products, key=by_price))
ranked = sorted(products, key=by_price, reverse=True)
print([name for name, _ in ranked])


def make_discount(rate):
    def apply(price):
        return round(price * (1 - rate), 2)
    return apply


print(make_discount(0.1)(300))
```

OUTPUT

```
('mouse', 25)
['laptop', 'monitor', 'mouse']
270.0
```

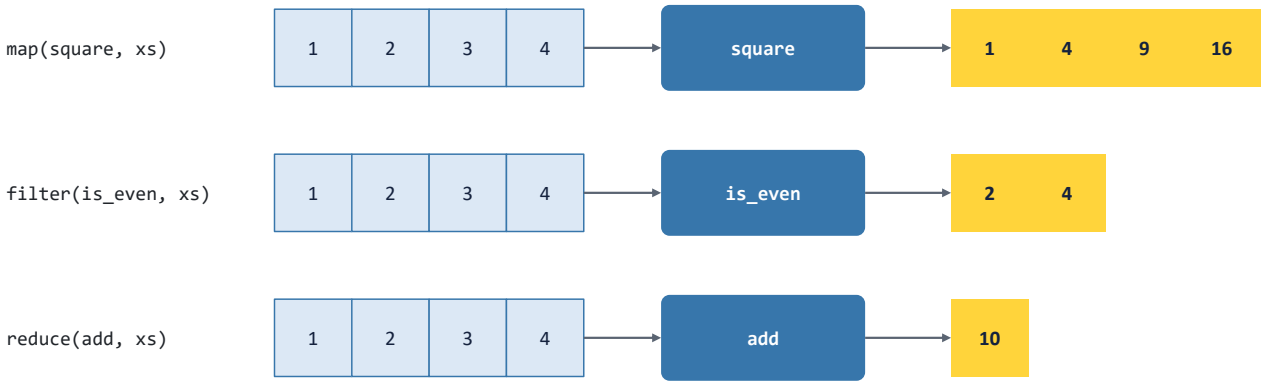

04

Map, Filter, Reduce and Comprehensions

Transforming collections declaratively

- Map, filter and reduce
- List, dict and set comprehensions
- Generator expressions and built-in reductions

Map, Filter and Reduce at a Glance



Key idea

`map` and `filter` return **lazy iterators**; `reduce` lives in `functools` and returns a single value.

filter and map in Action

```
>>> numbers = [4, 8, 15, 16, 23, 42]
>>> def is_odd(number):
...     return number % 2 == 1
...
>>> list(filter(is_odd, numbers))
[15, 23]
>>> odd = filter(is_odd, numbers)
>>> squares = map(lambda n: n * n, odd)
>>> squares
<map object at 0x...>
>>> list(squares)
[225, 529]
>>> list(squares)
[]
```

- `filter(predicate, iterable)` keeps the items for which the predicate is true
- `map(function, iterable)` applies the function to every item
- Both return **lazy iterators**: nothing runs until you consume them

Common misconception

An iterator can be consumed only once: the second `list(squares)` is empty.

reduce: Folding Values into One

```python
from functools import reduce

numbers = [4, 8, 15, 16, 23, 42]
odd = filter(lambda n: n % 2 == 1, numbers)
squares = map(lambda n: n * n, odd)
total = reduce(lambda acc, n: acc + n, squares, 0)
print(total)
print(sum(n * n for n in numbers if n % 2 == 1))
```

OUTPUT

```
754
754
```

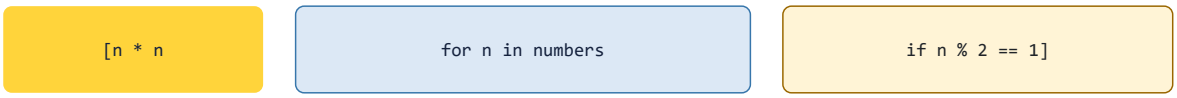
`reduce(function, iterable, initial)` calls `function(acc, item)` for each item



Tip

For totals use `sum`; use `reduce` only when no built-in reduction fits.

List Comprehensions



expression: *what* to produce — for clause: *where* items come from — optional filter: *which* items

```
>>> numbers = [4, 8, 15, 16, 23, 42]
>>> [n for n in numbers if n % 2 == 1]
[15, 23]
>>> [n * n for n in numbers if n % 2 == 1]
[225, 529]
>>> [word.upper() for word in ["map", "zip"]]
['MAP', 'ZIP']
```

With functions	As a comprehension
`list(map(f, xs))`	`[f(x) for x in xs]`
`list(filter(p, xs))`	`[x for x in xs if p(x)]`

Tip

Prefer a comprehension whenever the alternative needs a `lambda`.

Dict, Set and Generator Comprehensions

```
>>> fruits = ["apple", "fig", "kiwi", "plum"]
>>> {fruit: len(fruit) for fruit in fruits}
{'apple': 5, 'fig': 3, 'kiwi': 4, 'plum': 4}
>>> {len(fruit) for fruit in fruits}
{3, 4, 5}
>>> lengths = (len(fruit) for fruit in fruits)
>>> lengths
<generator object <genexpr> at 0x...>
>>> list(lengths)
[5, 3, 4, 4]
```

Syntax	Builds
`[expr for x in xs]`	list
`{key: value for x in xs}`	dict
`{expr for x in xs}`	set
`(expr for x in xs)`	generator (lazy)

Watch out

`{}` is an empty **dict**; an empty set is written `set()`.

Reductions: sum, min, max, any and all

```
>>> orders = [120, 35, 250, 80]
>>> sum(order for order in orders if order > 50)
450
>>> min(orders), max(orders)
(35, 250)
>>> any(order > 200 for order in orders)
True
>>> all(order > 50 for order in orders)
False
>>> sum(1 for order in orders if order > 100)
2
```

Goal	Prefer
Build a list, dict or set	comprehension
Aggregate to one value	`sum`, `min`, `max`, `any`, `all`
Apply an existing named function	`map(func, xs)` is fine
No built-in reduction fits	`functools.reduce`
Large or endless data	generator expression

Tip

any and `all` **short-circuit**: they stop at the first decisive element.

TRACE THE CODE

Predict the Comprehension Results

```
1  words = ["map", "filter", "reduce", "zip"]
2  lengths = {word: len(word) for word in words}
3  short = [word.upper() for word in words if len(word) <= 3]
4  total = sum(len(word) for word in words)
5  pairs = [(word, len(word)) for word in words if "e" in word]
6
7  print(lengths["filter"])
8  print(short)
9  print(total)
10 print(pairs[-1])
11 print(any(word.startswith("r") for word in words))
```

- Predict the five printed lines
- Which lines build a collection, and which aggregate?

ANSWER

Predict the Comprehension Results

OUTPUT

```
6
['MAP', 'ZIP']
18
('reduce', 6)
True
```

- Line 2 maps each word to its length
- Line 3 filters first, then transforms
- Line 4: 3 + 6 + 6 + 3, no list built
- Line 5 keeps words containing `e`
- `any` stops at the first match

Key idea

Lines 2, 3 and 5 **build** collections (dict, list, list); line 4 and the `any` call only **aggregate**, so a generator expression is enough and no list is built.

05

Iterators and Generators

Producing values lazily, one at a time

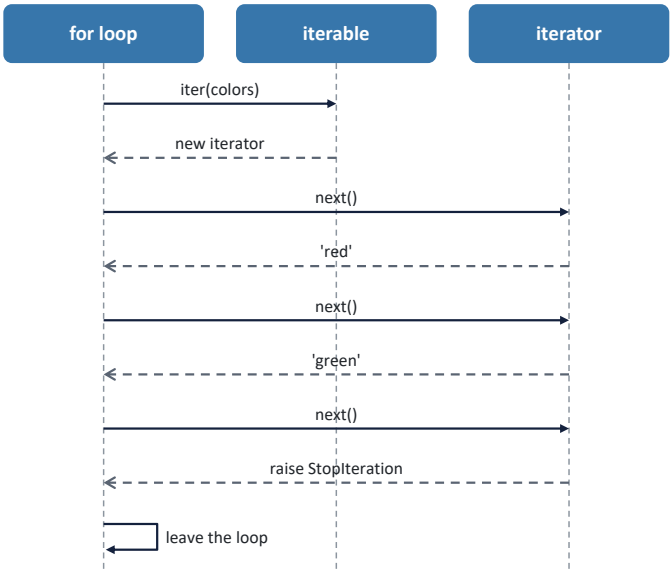
- The iteration protocol: iter, next and StopIteration
- Generator functions and yield
- Lazy pipelines, yield from and itertools

Iterables and Iterators

	Iterable	Iterator
Special methods	`__iter__`	`__next__` and `__iter__`
Built-in call	`iter(x)` gives an iterator	`next(it)` gives the next value
Examples	`list`, `str`, `dict`, `range`	`map`, `filter`, generators, files
Passes	many: a new iterator each time	one: then it is exhausted

```
>>> colors = ["red", "green"]
>>> iterator = iter(colors)
>>> next(iterator)
'red'
>>> next(iterator)
'green'
>>> next(iterator)
Traceback (most recent call last):
  ...
StopIteration
```

The Iteration Protocol



`for color in colors: print(color)` does:

```
colors = ["red", "green"]
iterator = iter(colors)
while True:
    try:
        color = next(iterator)
    except StopIteration:
        break
    print(color)
```

OUTPUT

```
red
green
```


Generator Functions

Iterator class: the protocol by hand

```
class Countdown:
    def __init__(self, start: int) -> None:
        self.current = start

    def __iter__(self) -> "Countdown":
        return self

    def __next__(self) -> int:
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1


print(list(Countdown(3)))
```

OUTPUT

```
[3, 2, 1]
```

Generator function: `yield` does it all

```
from collections.abc import Iterator


def countdown(start: int) -> Iterator[int]:
    while start > 0:
        yield start
        start -= 1


print(list(countdown(3)))
```

OUTPUT

```
[3, 2, 1]
```

- A function containing `yield` is a **generator function**
- Calling it returns a **generator object**, an iterator
- Use it in a `for` loop; values are computed lazily

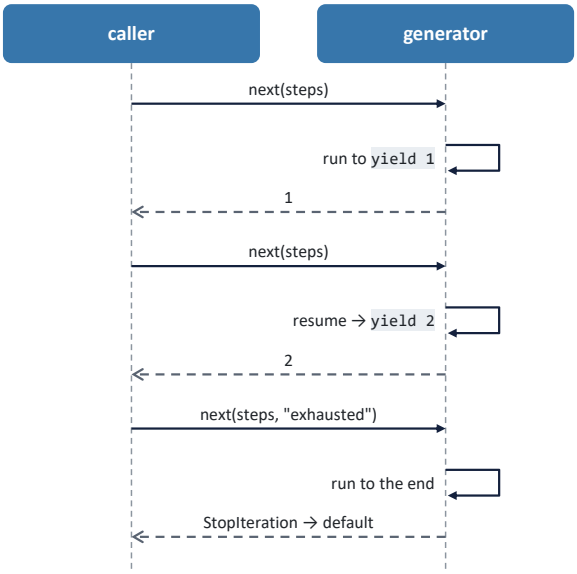
How a Generator Pauses and Resumes

```python
def two_steps():
    print("-> started")
    yield 1
    print("-> resumed after 1")
    yield 2
    print("-> resumed after 2")


steps = two_steps()  # nothing runs yet
print(next(steps))
print(next(steps))
print(next(steps, "exhausted"))
```

OUTPUT

```
-> started
1
-> resumed after 1
2
-> resumed after 2
exhausted
```



Example: The Collatz Sequence

collatz.py

```python
from collections.abc import Iterator


def collatz(n: int) -> Iterator[int]:
    """Yield the Collatz sequence after n, ending with 1."""
    while n > 1:
        n = n // 2 if n % 2 == 0 else 3 * n + 1
        yield n


steps = collatz(7)
print(next(steps), next(steps), next(steps))
print(list(steps))
print(max(range(1, 30), key=lambda n: len(list(collatz(n)))))
```

OUTPUT

```
22 11 34
[17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1]
27
```

Lazy Evaluation Saves Memory

Eager: list

- Computes **all** values immediately
- Memory grows with the data
- Many passes, `len()`, indexing

Lazy: generator

- Computes each value **on request**
- Small, constant memory
- One pass; may even be infinite

```python
import sys

eager = [n * n for n in range(1_000_000)]
lazy = (n * n for n in range(1_000_000))

print(sys.getsizeof(eager))  # bytes
print(sys.getsizeof(lazy))
print(sum(eager) == sum(lazy))
```

SAMPLE OUTPUT (CPYTHON 3.14, 64-BIT)

```
8448728
208
True
```


Generator Pipelines

```python
from collections import Counter

LOG = ["INFO start", "ERROR disk full", "INFO retry",
       "ERROR disk full", "ERROR timeout"]

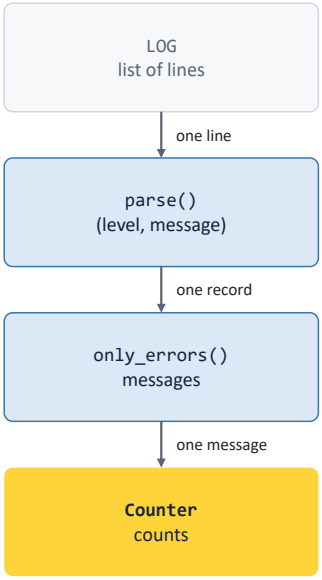

def parse(lines):
    for line in lines:
        level, _, message = line.partition(" ")
        yield level, message


def only_errors(records):
    for level, message in records:
        if level == "ERROR":
            yield message


print(Counter(only_errors(parse(LOG))))
```

OUTPUT

```
Counter({'disk full': 2, 'timeout': 1})
```



Delegating with yield from

```python
from itertools import chain


def flatten(nested):
    """Yield the items of arbitrarily nested lists."""
    for item in nested:
        if isinstance(item, list):
            yield from flatten(item)
        else:
            yield item


print(list(flatten([1, [2, [3, 4]], 5])))
print(list(chain("ab", [1, 2], range(3))))
```

OUTPUT

```
[1, 2, 3, 4, 5]
['a', 'b', 1, 2, 0, 1, 2]
```

- `yield from iterable` yields every value of another iterable
- Shorter than a `for` loop with `yield` inside
- Ideal for **recursive** generators: trees, nested data
- `itertools.chain` concatenates iterables lazily

Infinite Generators with count and islice

```python
from itertools import count, islice


def fibonacci():
    """Yield the Fibonacci numbers forever."""
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b


print(list(islice(fibonacci(), 10)))
print(list(islice(count(100, 10), 3)))
evens = (n for n in count() if n % 2 == 0)
print(next(evens), next(evens), next(evens))
```

OUTPUT

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
[100, 110, 120]
0 2 4
```

- `count(start, step)` counts forever
- `islice(iterable, n)` takes the first `n` values, lazily
- The **consumer** decides how much work is done

> **Watch out**
> `list(fibonacci())` never returns: always bound an infinite generator.

itertools: groupby and accumulate

```
>>> from itertools import accumulate, groupby
>>> sales = [3, 1, 4, 1, 5]
>>> list(accumulate(sales))
[3, 4, 8, 9, 14]
>>> list(accumulate(sales, max))
[3, 3, 4, 4, 5]
>>> [(k, len(list(g))) for k, g in groupby("aaabcc")]
[('a', 3), ('b', 1), ('c', 2)]
>>> [k for k, _ in groupby("abba")]
['a', 'b', 'a']
```

Function	Yields
`accumulate(xs, f)`	running results of `f`
`groupby(xs, key)`	`(key, group)` per run of equal keys
`pairwise(xs)`	overlapping pairs
`batched(xs, n)`	tuples of `n` items (3.12+)

Watch out

`groupby` only groups **adjacent** items: sort by the same key first.

QUIZ

Quick Check: Reusing a Generator

What does this program print?

```
squares = (n * n for n in range(4))
print(sum(squares))
print(sum(squares))
```

A `14` then `14`

B `14` then `0`

C `14` then a `StopIteration` traceback

D `0` then `14`

ANSWER

Quick Check: Reusing a Generator

What does this program print?

```
squares = (n * n for n in range(4))
print(sum(squares))
print(sum(squares))
```

A `14` then `14`

B 14 then `0`

C `14` then a `StopIteration` traceback

D `0` then `14`

Why

The first `sum` consumes the generator (0 + 1 + 4 + 9). The exhausted generator yields nothing more, so the second `sum` returns its start value `0`.

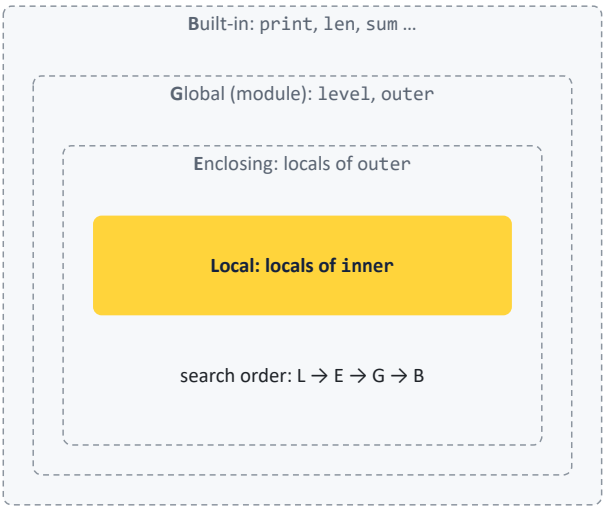
06

Closures and Decorators

Functions that remember and functions that wrap

- Scopes and the LEGB rule
- Closures and nonlocal
- Decorators, functools.wraps and decorator arguments

Scopes and the LEGB Rule




```
level = "global"


def outer():
    level = "enclosing"

    def inner():
        print(level)  # found in E
    inner()


outer()
print(level)
print(len(level))  # len found in B
```

OUTPUT

```
enclosing
global
6
```

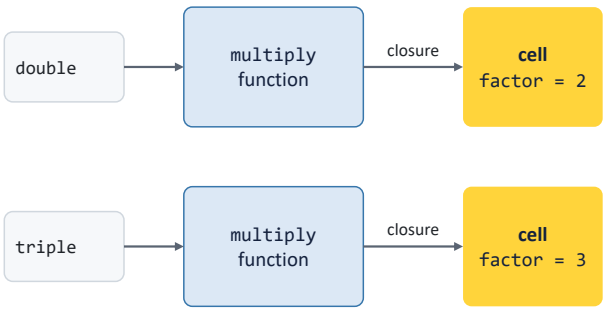

Closures: Functions That Remember

```
def make_multiplier(factor):
    def multiply(value):
        return value * factor
    return multiply


double = make_multiplier(2)
triple = make_multiplier(3)
print(double(10), triple(10))
print(double.__closure__[0].cell_contents)
```

OUTPUT

```
20 30
2
```



the frames of `make_multiplier` are gone; the cells survive

- A **closure** is a function plus the variables it captured
- Captured variables live in **cells** that outlive the call

Changing Captured State with nonlocal

```
def make_counter():
    count = 0

    def increment():
        nonlocal count
        count += 1
        return count
    return increment


counter = make_counter()
counter()
counter()
print(counter())
```

OUTPUT

```
3
```

```
>>> def make_broken_counter():
...     count = 0
...     def increment():
...         count += 1
...         return count
...     return increment
...
>>> make_broken_counter()()
Traceback (most recent call last):
  ...
UnboundLocalError: cannot access local variable ...
```

Key idea

Assignment makes a name local. `nonlocal` rebinds the enclosing variable; `global` rebinds a module variable.

TRACE THE CODE

Late Binding in Closures

```
1  multipliers = [lambda x: x * i for i in range(3)]
2  print([multiply(10) for multiply in multipliers])
3
4  fixed = [lambda x, i=i: x * i for i in range(3)]
5  print([multiply(10) for multiply in fixed])
```

- Predict both lines of output
- Why do the two lines differ?

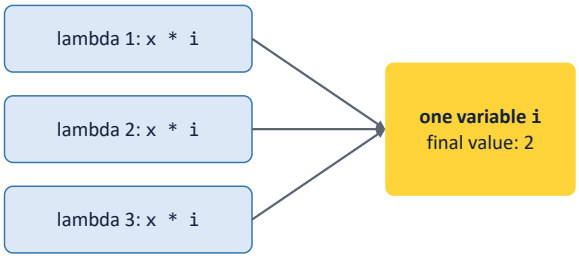
ANSWER

Late Binding in Closures

OUTPUT

```
[20, 20, 20]
[0, 10, 20]
```

- Closures capture **variables**, not values
- All three lambdas share one `i`, which is 2 when they run
- A default value is evaluated when each lambda is **created**



6 · CLOSURES AND DECORATORS

Decorators: Wrapping a Function

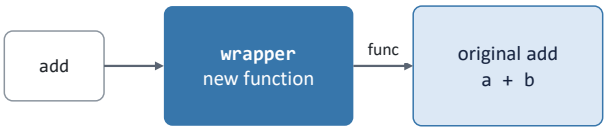
```python
def log_calls(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print(f"{func.__name__}{args} = {result}")
        return result
    return wrapper


@log_calls
def add(a, b):
    return a + b


total = add(2, 3)
```

OUTPUT

```
add(2, 3) = 5
```





Key idea

`@log_calls` above `def add` means `add = log_calls(add)`.

Preserving Metadata with functools.wraps

```python
import functools


def log_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print(f"{func.__name__}{args} = {result}")
        return result
    return wrapper


@log_calls
def add(a, b):
    """Return a + b."""
    return a + b


print(add.__name__, "|", add.__doc__)
```

OUTPUT

```
add | Return a + b.
```

Attribute	Without	With `wraps`
`__name__`	`'wrapper'`	`'add'`
`__doc__`	None	`'Return a + b.'`
`__wrapped__`	missing	original `add`

Tip

Always use `functools.wraps`: `help()`, tracebacks, pytest and FastAPI rely on this metadata.

Decorators with Arguments

```python
import functools


def tag(name):
    """Wrap the result in <name>...</name>."""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            text = func(*args, **kwargs)
            return f"<{name}>{text}</{name}>"
        return wrapper
    return decorator


@tag("b")
@tag("i")
def greet(user):
    return f"hi {user}"


print(greet("ada"))
```



OUTPUT

```
<b><i>hi ada</i></b>
```

Key idea

Stacked decorators apply bottom-up: `greet = tag("b")(tag("i")(greet))`.

Decorators You Already Use

Decorator	What it does	Module
`@property`	turns a method into a computed attribute	1
`@classmethod`, `@staticmethod`	change how a method is bound	1
`@dataclass`	class decorator: generates `__init__`, `__repr__`, `__eq__`	1
`@functools.cache`, `@lru_cache(maxsize=…)`	memoize results of pure functions	2, 11
`@contextlib.contextmanager`	builds a context manager from a generator	4
`@pytest.fixture`, `@pytest.mark.parametrize`	register test helpers and parameters	3
`@app.get("/books")`	registers a function as a route	8

Note

A decorator is any callable applied at definition time: it may wrap, register or transform the function or class.

TRY IT

Write a count_calls Decorator

Write `count_calls`, which counts the calls of the decorated function and exposes the count as the attribute `calls`:

```python
@count_calls
def ping():
    return "pong"


ping()
ping()
print(ping.calls, ping.__name__)  # 2 ping
```

Tip

Store the counter on the wrapper (`wrapper.calls`) and do not forget `functools.wraps`.

ANSWER

Write a count_calls Decorator

```python
import functools


def count_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        wrapper.calls += 1
        return func(*args, **kwargs)
    wrapper.calls = 0
    return wrapper


@count_calls
def ping():
    return "pong"


ping()
ping()
print(ping.calls, ping.__name__)
```

OUTPUT

```
2 ping
```

- `count_calls` runs once: one `wrapper`, one counter
- Every call increments `wrapper.calls`
- The name `ping` now refers to `wrapper`
- `functools.wraps` keeps the name `ping`

07

The functools Toolbox

Ready-made higher-order functions and decorators

- partial: pre-fill arguments
- cache and lru_cache: memoization
- cached_property: compute once per instance

functools.partial: Pre-Filling Arguments

```python
from functools import partial


def power(base: float, exponent: float) -> float:
    return base ** exponent


square = partial(power, exponent=2)
cube = partial(power, exponent=3)
print(square(5), cube(2))
print(square.func.__name__, square.keywords)
```

OUTPUT

```
25 8
power {'exponent': 2}
```

```
>>> from functools import partial
>>> parse_binary = partial(int, base=2)
>>> parse_binary("1010")
10
>>> print_csv = partial(print, sep=",")
>>> print_csv("ada", 36, "London")
ada,36,London
```

Key idea

`partial(f, *args, **kwargs)` returns a new callable with those arguments fixed.

Memoization with cache and lru_cache

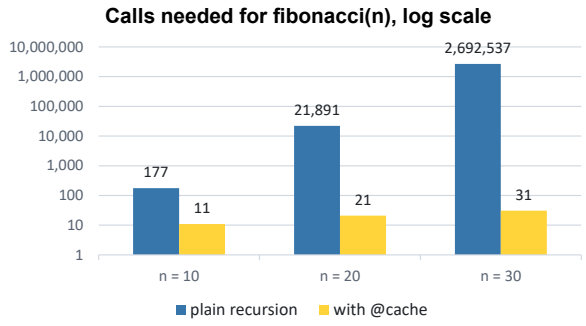
```python
from functools import cache


@cache
def fibonacci(n: int) -> int:
    """Return the n-th Fibonacci number."""
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)


print(fibonacci(80))
print(fibonacci.cache_info())
```

OUTPUT

```
23416728348467685
CacheInfo(hits=78, misses=81, maxsize=None, currsize=81)
```



Tip

`lru_cache(maxsize=128)` bounds memory; `cache` (3.9+) is unbounded. Cache only pure functions with hashable arguments.

cached_property: Compute Once per Instance

```python
from functools import cached_property


class Report:
    def __init__(self, values: list[float]) -> None:
        self.values = values

    @cached_property
    def statistics(self) -> dict[str, float]:
        print("computing...")
        mean = sum(self.values) / len(self.values)
        return {"mean": mean, "peak": max(self.values)}


report = Report([4, 8, 15, 16, 23, 42])
print(report.statistics["mean"])
print(report.statistics["peak"])
```

OUTPUT

```
computing...
18.0
42
```

- First access runs the method and stores the result on the instance
- Later accesses are plain attribute look-ups
- `del report.statistics` clears the cached value
- Compare `@property`: recomputed on every access

QUIZ

Quick Check: Choose the functools Tool

You need `parse_hex(text)` that behaves like `int(text, base=16)`. What is the most direct tool?

A. `functools.partial(int, base=16)`

B. `functools.reduce(int, text, 16)`

C. `@functools.cache` applied to `int`

D. `functools.wraps(int)`

08

Case Study: A Functional Data Pipeline

Refactoring the employee report

- Model the data immutably
- Small, pure, well-named functions
- Compose with filter, map, sum, comprehensions and groupby

Case Study: The Employee Data Model

employees.py

```python
from typing import NamedTuple


class Employee(NamedTuple):
    name: str
    department: str
    salary: int
    born: int
    full_time: bool


EMPLOYEES = (
    Employee("jack shephard", "Sales", 100_000, 1978, True),
    Employee("kate austen", "IT", 200_000, 1985, False),
    Employee("ben linus", "Finance", 150_000, 1967, True),
    Employee("james sawyer", "HR", 70_000, 1979, False),
    Employee("kim kwon", "Sales", 120_000, 1986, True),
    Employee("sun kwon", "IT", 200_000, 1984, False),
    Employee("hugo reyes", "IT", 120_000, 1992, True),
)
```

The Original Solution Revisited

original version (slides 21–22)

```python
def to_salary(employee) -> float:
    return employee["salary"]


def if_part_time(employee) -> bool:
    return not employee["fulltime"]

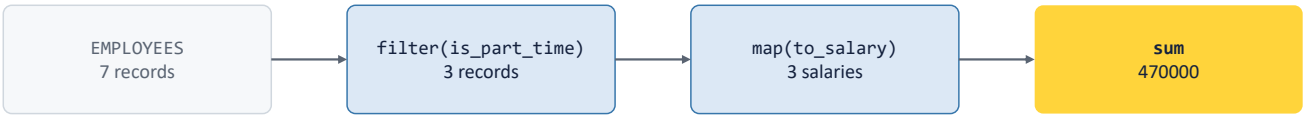

def gun(x, y, z):
    return x + y + z


def sun(x, y):
    return partial(gun, z=0)


print(reduce(partial(gun, z=0), map(to_salary, \
       filter(if_part_time, employees)), 0))
```

- Meaningless names: `gun`, `sun`
- `if_part_time` reads like a statement, not a predicate
- `sun` is never used and ignores its parameters
- `partial(gun, z=0)` is plain addition: use `sum`
- Nested calls read inside-out; backslash continuation
- Mutable dicts: key typos fail only at run time

Pipeline 1: Part-Time Payroll



report.py

```python
from employees import EMPLOYEES, Employee


def is_part_time(employee: Employee) -> bool:
    return not employee.full_time


def to_salary(employee: Employee) -> int:
    return employee.salary


part_time = filter(is_part_time, EMPLOYEES)
print(sum(map(to_salary, part_time)))
print(sum(emp.salary for emp in EMPLOYEES if not emp.full_time))
```

OUTPUT

```
470000
470000
```

Pipeline 2: Payroll per Department



report.py (continued)

```python
from itertools import groupby
from operator import attrgetter

from employees import EMPLOYEES

by_department = attrgetter("department")
ordered = sorted(EMPLOYEES, key=by_department)
payroll = {
    department: [emp.salary for emp in group]
    for department, group in groupby(ordered, key=by_department)
}
for department, salaries in payroll.items():
    print(f"{department:<8}{len(salaries):>2}{sum(salaries):>9,}")
```

OUTPUT

```
Finance  1  150,000
HR       1   70,000
IT       3  520,000
Sales    2  220,000
```


MINI LAB

Extend the Employee Report

1. Average salary per department, highest average first
2. Names of full-time employees born before 1980, in *Title Case*, sorted
3. The highest-paid employee, using `max` and `attrgetter`
4. A generator `batches(items, size)` yielding tuples of `size` employees
5. A `@timed` decorator (with `functools.wraps`) for the report functions

Tip

Work in `report.py` and import `EMPLOYEES` from `employees.py`. Keep every helper pure.

MODULE 02 · SUMMARY

Key Takeaways

- ✓ The call stack pushes a frame per call; locals live and die with their frame
- ✓ Pure functions and immutable data make code predictable, testable and cacheable
- ✓ Functions are first-class: pass them (`key=`, `map`, `filter`), return and store them
- ✓ Comprehensions and generator expressions are the idiomatic way to transform data
- ✓ Generators produce values lazily; `itertools` builds memory-efficient pipelines
- ✓ Closures remember their scope; decorators wrap functions, always with `functools.wraps`
- ✓ `functools` adds `partial`, `reduce`, `cache`/`lru_cache` and `cached_property`

NEXT UP Module 03 — Unit Testing in Python with `pytest`